Highlights

From detection to action: closing the autonomic loop of a self-coordinated manufacturing line with rule-based and retrieval-augmented LLM planners

Manuel Sánchez, Ernesto Exposito

- Our earlier autonomic architecture detected conditions but never planned or acted.
- Its alarms are sparse line-wide bursts; with them no planner recovers delay.
- A per-station change test on processing times makes closed-loop repair pay off.
- A frontier LLM vetoing the test's candidates leads where false repairs are costly.
- A burst-veto rule gets most of that gain; with common-cause faults both lose.

From detection to action: closing the autonomic loop of a self-coordinated manufacturing line with rule-based and retrieval-augmented LLM planners

Manuel Sánchez[a,b,*], Ernesto Exposito[b]

[a] *Universidad Nacional Experimental del Táchira, San Cristóbal, Venezuela*

[b] *Univ Pau & Pays Adour / E2S UPPA, Laboratoire d'informatique de l'Université de Pau et des Pays de l'Adour (LIUPPA), EA3000, 64000 Pau, France*

Abstract

Autonomic architectures for Industry 4.0 promise self-healing production, but published systems often stop at detection. We revisit our earlier self-coordinated manufacturing architecture, find that it implements Monitor and Analyze but not Plan, Execute or a Knowledge base, port its simulator to a tested headless engine over the Bosch production-line dataset, and close the loop with an action space built on the line's parallel twin stations, a symbolic validator, synthetic faults and an explicit effect model. We compare rule-based planners with large-language-model (LLM) planners, with and without retrieved operational statistics, on a local 7B model (Qwen2.5) and a frontier model (Claude Fable 5.1), with parameters and prompts fixed on a development trace, on 216 fault scenarios from three held-out traces and 360 from five traces generated after the design was frozen, by the net delay recovered within 24 h of the fault. The original Analyze layer is the bottleneck: its alarms come in line-wide bursts, and with it no planner recovers more than 7% of the delay. With a per-station change test on processing times, introduced after that failure was seen, a frontier LLM that vetoes the test's candidates recovers 41% (95% interval 32–50%) on the held-out traces and 49% (42–55%) on the new ones, against 20% and 17% for the best rule with default settings; three replications without stored responses give 42–43%. Much of the gain comes from not acting on alarms shared by many stations: a rule that vetoes such alarms recovers 34% and 40%,

[*] Corresponding author.

Email addresses: mbsanchez@unet.edu.ve (Manuel Sánchez), ernesto.exposito@univ-pau.fr (Ernesto Exposito)

and the LLM's margin over it is 8–9 points. How much the veto is worth depends on what a false repair costs. With long repairs, a stopped line, full congestion or mild faults, the LLM hybrid loses far less than any rule; with severe faults, short repairs or no congestion, all planners are level; and when one cause degrades four stations at once, the veto discards real faults and the hybrid falls behind the plain rule. A first evaluation design, with the fault injected at a fixed time inside the detector's start-up burst, had produced apparent gains that disappeared once the design was corrected.

Keywords: Autonomic computing, MAPE-K, Industry 4.0, Large language models, Retrieval-augmented generation, Fault management

---

1. Introduction

Autonomic computing proposes systems that manage themselves through a Monitor–Analyze–Plan–Execute loop over a shared Knowledge base (MAPE-K) [1, 2]. In manufacturing, this loop is attractive because production lines degrade continuously and human supervision does not scale: a self-healing line would notice a degrading station and reconfigure itself before the delay accumulates.

In earlier work we proposed an architecture for self-coordinated manufacturing processes with self-* properties and evaluated it with a simulator, SSACSimulator, over the Bosch production-line dataset [3, 4]. Revisiting that code, we found that it implements the Monitor and Analyze phases only. When one of its three detectors fires, the simulator writes *"launching the autonomous cycle for reconfiguration"* to a log panel; no plan is produced and nothing is executed. The loop was announced, not closed.

This paper closes it, and asks what closing it actually requires. Two developments make the question timely. First, large language models (LLMs) are increasingly proposed as planners for industrial decision making, from the control of modular production systems [5] to advanced planning and scheduling [6]. Second, retrieval-augmented generation (RAG) [7] offers a natural candidate for the Knowledge component of MAPE-K: instead of a hand-built ontology, the planner retrieves the operational history it needs. Whether either helps, against which baseline, and under which Analyze layer, is an empirical question.

Our contributions are:

1. A faithful, tested, headless port of the original simulator [3], and an analysis of its Analyze layer on the published data: one detector is

unreachable, one is exercised by 0.004% of the events, and the third raises line-wide bursts of alarms, concentrated at the start of each trace, that name a faulty station within a day in only half of the scenarios (Sections 3 and 7.1).

2. The missing Plan, Execute and Knowledge phases, and a per-station Analyze on processing times: an action space based on the parallel "twin" stations of the Bosch line, a symbolic validator, fault injection with three fault shapes, and an explicit, parameterised effect model (Section 4).
3. A comparison of rule-based planners against LLM planners with and without retrieved knowledge, and of a hybrid in which a statistical test proposes and the LLM vetoes, on a local 7B model and a frontier model, under both Analyze layers, with every parameter and prompt chosen on a development trace, 216 fault scenarios on three held-out traces and 360 on five traces generated after the design was frozen (Sections 5–7).
4. Evidence that the Analyze layer sets what any planner can recover and the planner how much of it is recovered; that, given useful alarms, a capable LLM adds most as a veto over a statistical test, by avoiding repairs that cost more than they fix; that a plain rule vetoing alarm bursts reproduces most of that gain; that the veto is worth much where false repairs are costly, little where they are cheap, and harmful under common-cause faults; and that a fixed fault onset can manufacture conclusions that disappear under a sounder design (Sections 7 and 8).

All code, data-processing scripts, prompts, every model decision with its stated reason, and all results are available (Section Appendix A).

2. Background and related work

Autonomic computing and self-adaptive systems. The MAPE-K reference model [1] structures a self-managing system as a feedback loop around a managed resource. Engineering such loops is the subject of the self-adaptive systems literature [2], which stresses that the Analyze phase must separate genuine change from noise before any adaptation is planned. LLMs have recently been examined as components of such loops: a state-of-the-art review and research roadmap organises their potential uses around the MAPE-K functions [8, 9]; LLM-based agents have been embedded in MAPE-K-based multi-agent systems [10]; and MAPER extends MAPE-K with an LLM-based reasoning component to handle unanticipated runtime situations [11].

Our study differs in placing an LLM inside the loop of an industrial system whose detector we first inherit unchanged and then replace, and in measuring it against tuned non-LLM baselines on held-out data and on data generated after the design was frozen.

Self- properties in manufacturing.* Our earlier work integrated multi-agent systems, service orientation and autonomic computing for Industry 4.0 process coordination [3, 4, 12]. The present paper re-examines that architecture empirically rather than proposing a new one.

Alarm management and change detection. In process industries, alarm management standards require alarms to be rationalised, so that each one is meaningful and calls for a response, before operators are asked to act on them [13]; a technical report applies the same principles to batch and discrete manufacturing [14], the setting of a line such as Bosch's. An autonomic controller that acts on alarms faces the same requirement, without an operator to absorb the noise. Detecting a change in the rate of an event stream or in the distribution of a measurement is a classical problem [15]; our rule-based planners and our per-station detector are simple instances of it.

LLMs and retrieval in industrial decision making. RAG conditions a language model on documents retrieved for each query [7]. In manufacturing, LLM agents have been combined with digital twins to plan and control modular production [5] and applied to advanced planning and scheduling [6], and a recent review of LLMs for fault diagnosis in smart manufacturing covers, among other roles, retrieval-augmented fusion of industrial knowledge [16], and another reviews how humans are kept in the loop when LLMs are used in smart manufacturing [17]. Here, the retrieved knowledge is structured operational statistics computed by code from the controller's own history, not documents; we keep the term RAG for the retrieval step and state exactly what is retrieved (Section 5).

3. The original system revisited

3.1. Simulator and data

SSACSimulator is a Qt/C++ discrete-event simulator. A process model, derived from the Bosch Production Line Performance dataset [18], describes 52 stations on four sub-lines plus begin and end nodes, 285 transitions (84 of them main flows), a throughput time of 6 420 min and 1 183 165 cases. Four event traces of 100 000 transitions each, prepared for the original study [3],

replay recorded piece movements; they span 54 to 168 hours of production. For validation we cut five more traces from the same raw files, after the design was frozen (Section 6). All traces come from the dataset's test set: its 1 183 748 pieces, less 583 without any timestamp, give the 1 183 165 cases of the process model. The events of all these pieces are sorted by time and cut into consecutive chunks of 100 000, so each trace is a contiguous period of production that contains every test-set piece active in it, but not the pieces of the dataset's training set, which the original study did not use. Timestamps are anonymised; as in the original study, one unit of the dataset is taken as 600 min, so that its resolution of 0.01 units gives the 6-min granularity of the traces. Production is recorded in batches: 62 of the 157 hours of trace 100m–200m contain no event.

3.2. Headless port

We ported the simulation core (about 1 440 lines of C++ logic, excluding the graphical user interface) to Python. The port reproduces the original's behaviour as read from its source, including details that change results: thresholds computed in single precision, a 100 000-event loading limit, and a lenient rule for malformed rows. A fast path advances only the pieces that can leave the system; the original per-tick loop is kept as a test oracle, and a regression test checks that both produce identical detections, decisions and piece lifetimes. Parity is with the original as read from its source, not with the compiled Qt binary. A run takes 0.2–0.4 s.

3.3. What the Analyze layer actually does

The original implements three detectors: a station whose mean transition time exceeds 1.3 times its reference; more than 10% of stations flagged within one batch of simultaneous events; and a mean piece lifetime exceeding 1.1 times the throughput time. Only the second fires. The predicted-failure detector depends on a classifier label that is positive in 15 of 400 000 events across all traces (0.004%). The global detector is unreachable: the mean piece lifetime (636 min) is an order of magnitude below its threshold (7 062 min). Finally, 45 of the 285 transitions have a reference time of zero, so any positive duration flags their station. Section 7.1 shows what the one active detector sees once faults are injected.

Plan, Execute and Knowledge are absent. This is the starting point for the rest of the paper.

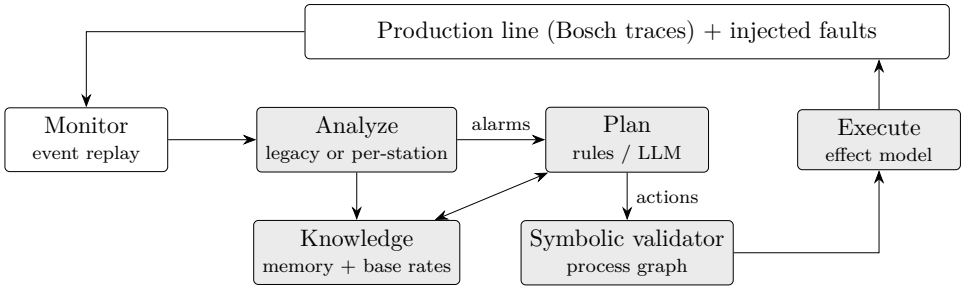


Figure 1: The closed MAPE-K loop. White: present in the original system [3]. Grey: added or replaced in this work. The Analyze layer is either the legacy detector or the per-station detector of Section 4.4.

4. Closing the loop

Figure 1 summarises the closed loop. Components added in this work are shaded.

4.1. Twin stations and the action space

The Bosch line is built from parallel station pairs (and a few triples) that share predecessors and successors and split the load almost evenly; for example, on their main flows stations L0_S2 and L0_S3 receive 339 345 and 334 054 cases. We identify twins automatically from the overlap of their predecessor and successor sets on main flows: 29 of the 52 stations have one. Diverting a piece from a station to its twin keeps its trajectory consistent with the process graph. Structural equivalence does not guarantee equivalent equipment or capacity, which the anonymised dataset does not describe; a plant would have to confirm each twin. The action space is Reroute(S, T, f), which diverts a fraction f of the pieces entering S to its twin T; Restore(S); Maintenance(S, d), which repairs S for d minutes; and NoOp.

4.2. Symbolic validator

Every action, from any planner, is checked against the process graph and the current interventions before execution: stations must exist, a reroute target must be a structural twin, reroutes must not form chains, and a station cannot be repaired twice concurrently. The validator never judges whether an action is a good idea, only whether it is admissible.

4.3. Effect model

The simulation is trace-driven: start times come from the recorded data and are not shifted; faults and actions act on transition durations, and the lifetime of a piece is the sum of its transition durations. Delay therefore does not propagate as queueing; the model estimates time lost per piece. A Bosch "transition" is the time between two station timestamps and mostly measures waiting: its median is 0 min and its 90th percentile 18–30 min, but its 99th percentile reaches 4 100–5 900 min. Effects are therefore *additive* and proportional to each station's processing time τ_S (the median of its transitions of up to 60 min), never multiplicative over a transition that includes days of storage. τ_S is 6 or 12 min for the eight stations where faults are injected, identical on all traces. The effect model is:

- **Fault** of factor k on S: $+(k(t) - 1)\,\tau_S$ on every transition leaving S, where $k(t)$ is a *step* (k from t_0), *intermittent* (k for 60 min, 1 for 120 min, repeating) or *drift* (rising linearly from 1 to k over 600 min).
- **Reroute**: a deterministic fraction of pieces (selected by a hash of the case identifier) goes to T, which then adds $\alpha\, f\, \tau_T$ for the extra load.
- **Maintenance**: faults on S already active when the repair starts are cleared when it ends; a fault that begins during a repair is not. An intermittent fault counts as active from its onset, also during its off phases. During the repair, in *contingency* mode (default) backup capacity outside the model processes the pieces, adding $0.5\,\tau_S$; in *stop* mode the pieces wait for the repair to finish. Repairing a healthy station costs and fixes nothing, which is what makes false alarms expensive.

Because the model is additive, the excess delay of a run decomposes exactly into the minutes added to individual transitions, which lets us measure it within any time window. The dataset is anonymised and provides no capacities, repair times or labelled failures, so all faults are synthetic and every parameter we could not ground in data (severity k, repair time d, congestion α, maintenance mode) is varied in a sensitivity analysis (Section 7.2).

4.4. A per-station Analyze on processing times

Section 7.1 shows that the legacy detector reports the injected faults rarely and only inside line-wide bursts. Its station test compares means of whole transitions, dominated by hours or days of storage, so a fault that adds minutes of processing barely moves them. We therefore add an alternative

Analyze layer that looks only at the processing regime: the transitions leaving a station that last at most 60 min. For each station, causally, it takes as reference the 75th percentile of those transitions over the previous 24 h and the rate p_0 at which they exceed it (Laplace-smoothed, because timestamps have a 6-min granularity); among the station's last N processing transitions, it counts those above the reference and flags the station when that count is improbable under Binomial(N, p_0) at level α. Nothing is flagged during an 8-h warm-up; the stations flagged in one batch of simultaneous events are reported in a single alarm, and a station is not flagged again within a refractory period. N, α and the refractory period were chosen on the development trace (Section 6): $N = 10$, $\alpha = 10^{-3}$ and 6 min. With this layer, the legacy station alarms are no longer passed to the planners. The planners themselves are unchanged.

5. Planners

All planners receive, for each alarm, the alarm and the state of their own past interventions. None sees the injected faults. All of them respond to a station judged faulty in the same way: reroute to a free twin if one exists, then repair. Every planner, including the LLM planners, leaves a station it has acted on alone for 240 min.

5.1. Rule-based planners

- **Naive**: acts on every station named in an alarm.
- **Calibrated**: acts when the number of alarms on a station within a 120-min window is improbable (Poisson tail, $\alpha = 10^{-3}$, at least 3 alarms) under its base rate, learned on a *different* fault-free trace.
- **Adaptive**: the same test, against the rate observed in the preceding 24 h of the same trace, causally, after an 8 h warm-up.
- **Adaptive + load veto** (control): the adaptive rule, ignoring stations that receive load the controller itself rerouted onto them.
- **Twin-only** (control): acts on every flagged station that has a free twin and ignores the others.
- **Burst veto** (control): the adaptive rule, rejecting the candidates of an alarm that names at least m stations, or of any alarm while at least m distinct stations have alarmed in the last W min; with or without the

load veto. As in the hybrid, a rejected station is not re-examined for 120 min. m and W were chosen on the development trace (Section 6): with the load veto, alarms of at least 3 stations; without it, at least 8 stations alarming within 120 min.

- **Random veto** (control): the adaptive rule, rejecting each candidate at random with the hybrid's per-candidate veto rate on the held-out traces (70%), with the same 120-min suspension; five seeds.

The controls are not proposed methods; they implement as plain rules behaviours observed in LLM planners, so that an LLM's contribution is measured beyond them. The burst veto is the reason the hybrid on Fable 5.1 gives most often for its vetoes (Section 7.7). The default window, level and history above were fixed before any evaluation; we also re-tune them on the development trace (Section 6).

5.2. LLM planners

For every alarm with actionable stations, the LLM must return, for each of them, a verdict (*degraded*, *normal*, *self-inflicted* or *insufficient evidence*) followed by an action type (none, repair, or reroute and repair). The output is constrained by a JSON schema whose station names are restricted to the actionable ones, so the decoder cannot invent a station. Code chooses the twin, discards any action inconsistent with the model's own verdict, and passes the rest to the validator.

- **C1** receives the alarm, the actionable stations, whether each has a free twin, and the current interventions: the same structural information as the rules.

- **C2 (RAG)** additionally receives knowledge retrieved from the controller's operational memory: for each station, the alarms observed in the last 2 and 24 h against the number its fault-free base rate predicts, whether the controller is pushing rerouted load onto it, whether its alarms dropped after past repairs, and its most recent log records. Retrieval is keyed by station and ordered by recency; all statistics are computed by code.

- **Hybrid**: the adaptive change test proposes the stations; the LLM, with C2's knowledge, confirms or vetoes and chooses the action. A vetoed station is not re-examined for 120 min. The prompt tells the model that the test is usually right but cannot see context, and gives

as examples load the controller rerouted onto a station, a past repair that did not reduce its alarms, and *"a burst of alarms shared by many stations at once"*.

The system prompt describes how the Analyze layer raises alarms. Prompt version v2 describes the legacy detector and was fixed on the development trace before any held-out run; v3, used with the per-station detector, differs from v2 only in that paragraph, and was written after the held-out runs with the legacy detector and tested on the development trace with the 7B model only (Section 6). Table 8 measures what that paragraph and the burst example change.

Models. We use Qwen2.5-7B-Instruct [19], 4-bit quantised, served locally with Ollama [20] at temperature 0; and Claude Fable 5.1 (Anthropic), accessed through the Claude Agent SDK [21] with effort set to *low* (SDK version 0.2.163). Each call replaces the harness's system prompt with the planner's, loads no settings or memory files (`setting_sources` empty, an empty temporary working directory), exposes no tools and no MCP servers, and enforces the planner's JSON schema as structured output. The stored usage is consistent with this: a call takes a median of 2 543 input tokens, for 4 136 characters of system prompt, request and schema. The frontier model does not expose a temperature setting, so its runs are not bit-reproducible. Every request and response of both models is stored, keyed by the full request. Identical requests are frequent, because before the fault onset a scenario yields the same states as the fault-free run of its trace, and they are served from this store: with the per-station detector, 2 403 of the 26 933 calls of C2 on Fable 5.1 were new. A new call takes a median of 5–8 s with either model, short against faults that cost hours. Each decision is a single sample of the model; to measure run-to-run variability we repeated the main hybrid run three times, each with its own key in the response store (Section 7.6). Fable 5.1 was used through a Claude subscription, with model identifier `claude-fable-5-1`, between 6 and 17 October 2026. A first development pilot was discarded because the SDK passed the tool definitions of the account's connectors to the model; the harness was then restricted to no tools and its identifier added to the cache key, so that no response from that pilot was reused. The prompts are given in Appendix A.

6. Experimental protocol

Scenarios. A fault of factor $k = 3$ is injected on one of eight stations, four with a twin (L0_S2, L0_S4, L0_S6, L3_S35) and four without (L0_S1,

L0_S8, L3_S29, L3_S33), all of which have observable processing times on every trace, with one of the three fault shapes, at one of three onsets per station and shape. Onsets are drawn uniformly between 24 h after the first event of the trace and 24 h before its last, from a generator seeded by the scenario's name. Each run stops 24 h after the onset. Each trace also has a fault-free run over its whole length. Repairs last 60 min, $\alpha = 0.5$, and maintenance runs in contingency mode unless stated otherwise.

Development and held-out traces. The first trace (0–100m) is the development trace; the other three are held out (216 fault scenarios). On the development trace only, we chose the parameters of the per-station detector (27 configurations of $N \in \{10, 20, 40\}$, $\alpha \in \{10^{-2}, 10^{-3}, 10^{-4}\}$ and refractory periods of 6, 30 and 60 min, selected by the recovery of the adaptive rule with load veto, so that the detector was tuned for a rule, not for the LLM planners), re-tuned the rules (45 configurations of window, level and history for each adaptive rule, 15 of window and level for the calibrated rule), and fixed the prompts. Every prompt version tried and its results are logged in the repository. Calibrated rules and C2 learn their base rates on a fault-free trace other than the one they are evaluated on (100m–200m from the development trace; 200m–300m and 500m–600m from each other). During prompt development, the development runs took their base rates from 100m–200m, which is later held out; this exposes no fault labels.

Sequence of decisions. The held-out traces were not untouched by the redesign, and we state how. The evaluation design (onsets, repair rule, horizon) was revised after a first evaluation on the held-out traces (Section 7.8). Under the revised design, the rules with the legacy detector recovered at most 4% on the held-out traces and the development trace had no legacy alarm after its first 24 h, so every rule configuration scored zero there; this is what led us to build the per-station detector. Before any run with it, we registered its form in the repository: the processing regime, the 24-h reference, the binomial test, the warm-up and a refractory period of 60 min, with N and α to be chosen on the development trace. We then added the refractory period to the grid, because with 60 min a station can never reach the three alarms in 120 min that the rules require; 6 min was chosen. Prompt v3 was written then, and tested on the development trace with the 7B model only. All held-out results with the per-station detector were obtained with these choices fixed. C1 on Fable 5.1 was run after the hybrid's held-out results were known. The mild-fault condition and the ablations of Table 8 were registered before they were run, after the main held-out results. So were,

later, the veto controls, the remaining sensitivity conditions, the common-cause faults, the replications and C2 on the validation traces; the controls' parameters were chosen on the development trace. A note on the refractory period, added after the fact, is in the same log.

Validation traces. To test the frozen design on data that informed no choice, we generated five further traces from the raw dataset [18] with a Python 3 port of the original preprocessing scripts, which reproduces the four existing traces row for row. We took, in order, the first five unused chunks of 100 000 events (300m–400m, 400m–500m, 600m–700m, 700m–800m and 800m–900m; 104 to 177 h each), with exclusion rules registered before generating them; none was excluded. Everything else was frozen: design, stations, shapes, three onsets per station and shape (360 fault scenarios), detector, prompts, models and response store. Base rates come from the development trace. The planners are the rules with default and re-tuned settings, the veto controls, the two hybrids and C2 on Fable 5.1; C1 was not run there.

Metrics. Every run replays the same trace, so each piece is compared with itself in a fault-free reference run. The *24-h recovery* of a planner over a set of scenarios is $1 - \sum E_{24}(\text{planner}) / \sum E_{24}(\text{no action})$, where E_{24} is the delay added to transitions in the 24 h after the fault onset: 1 erases the cost of the faults; 0 is no better than doing nothing; negative values mean that the planner's own actions cost more than the faults. The recovery is therefore net of the cost of acting. Because the effect model is additive, E_{24} splits exactly into the fault's own delay that remains (on transitions not covered by a repair or a reroute) and the delay caused by the planner's actions; we report this split on the scenarios in which the fault costs something within the window (Table 4). The window is chronological: production is recorded in batches, so a station may have few or no transitions in it. A planner *acts* on a scenario if it repairs the faulty station within those 24 h; *false repairs* are repairs of other stations in the same window. The *false-alarm cost* is the excess delay per hour in the fault-free runs.

Statistics. The primary estimand is the difference in 24-h recovery between two planners, with a 95% bootstrap interval [22] that resamples scenarios within their trace (10 000 resamples). Because scenarios on the same trace share their alarm noise and three traces are few clusters, we also report a two-level bootstrap that resamples traces and then scenarios, a bootstrap that resamples the 24 (held-out) or 40 (validation) clusters of trace and faulty station, because the 24-h windows of scenarios on the same trace overlap and share decisions, the difference with each trace left out, and

Table 1: What each Analyze layer reports about the faulty station in the 24 h after the fault onset, with no planner, on the 216 held-out fault scenarios, and in the same windows without the fault. Delay: median time from the onset to the first alarm naming the faulty station, in detected scenarios. Alarms per hour and stations per alarm: fault-free runs of the three held-out traces. The threshold control uses the same processing regime and windows as the per-station detector, with a fixed threshold instead of the binomial test.

Analyze layer	Scenarios with an alarm (fault / no fault)	Alarms per scenario (fault / no fault)	Delay (h)	Alarms per h without faults	Stations per alarm
Legacy detector	108 / 52	5.4 / 1.2	10.9	0.6	7.6
Per-station detector	122 / 107	15.1 / 3.6	3.5	1.6	2.0
Threshold control	134 / –	–	4.7	4.4	3.4

the difference in every ordered draw of the traces with replacement (27 for three traces, 3 125 for five). These are checks of stability over the traces we have, not estimates of how the result would generalise to other periods or lines. We report the number of scenarios in which one planner loses less time than the other and the matched-pairs rank-biserial correlation r as effect size; the Wilcoxon signed-rank tests on per-scenario delay [23], with Holm adjustment [24], treat scenarios as independent and are given in the repository only. The breakdown by fault shape is exploratory.

7. Results

7.1. The legacy Analyze layer gives too little evidence

The legacy station detector raises its alarms in bursts that follow the production batches rather than the faults. Without faults, on the held-out trace 100m–200m it raises 63 and 18 alarms in its first two 12-h blocks and at most 4 in each of the next twelve; on 200m–300m, 66 in the first 12 h, none in the next 60 h, then 25, 15 and 16 and at most 5 per block after that; on the development trace, 30, 9 and then none. Table 1 shows what it reports in the 24 h after a fault, with no planner. It names the faulty station in half of the held-out scenarios, a median of 10.9 h after the onset, 5.4 times on average against 1.2 in the same window without the fault, and each alarm that names it lists 6.5 stations on average, because the detector only fires when more than 10% of the stations are flagged in one batch. It compares mean transition times that are dominated by storage, so a fault that adds 12 min of processing moves them little.

With this Analyze layer, no planner recovers more than 7% within 24 h (Table 3, upper block): the calibrated and adaptive rules range from -0.03 to -0.02 (the twin-only control and C1 on the 7B model do much worse), C2 on Fable 5.1 reaches $+0.07$ $[-0.00, +0.14]$, and the hybrid on Fable 5.1,

which vetoes almost every candidate, +0.03. The planners have too little to act on.

7.2. A per-station Analyze layer

The per-station detector does not report the faulty station more often relative to its background than the legacy detector does: the ratio of alarms with and without the fault is 4.1 against 4.5, and it names a station that has no fault in half of the windows (Table 1). What it changes is when and how. It reports the faulty station a median of 3.5 h after the onset instead of 10.9 h, and in alarms that list 2.3 stations instead of 6.5 (Section 7.1), so that a rate test on its alarms can single the station out. The binomial test is what keeps it usable: the threshold control, with the same processing regime and windows, detects slightly more faults but raises 4.4 alarms per hour without faults instead of 1.6, and with it the rules fall to between −0.39 and −0.35. The binomial level is not a false-alarm rate: consecutive transitions of a station are correlated by batch and product, so the nominal $\alpha = 10^{-3}$ per test still yields 1.6 alarms per hour on a fault-free line. On the development trace, every configuration of the per-station detector names the faulty station within 24 h in all 72 development scenarios; the chosen one lets the adaptive rule with load veto recover +0.84 there.

Table 2 reports the rule-based planners with this detector on the held-out traces and a one-at-a-time sensitivity analysis over all fault shapes. Acting on the detector's alarms pays off only when faults are severe relative to the cost of a repair and backup capacity absorbs the work during repairs. Halving the fault factor ($k = 1.5$, which divides the delay a fault adds by four), quadrupling the repair time (240 min) or stopping the line during repairs makes every rule much worse than doing nothing, because the false repairs then cost more than the faults they fix; full congestion ($\alpha = 1$) cancels the benefit. Shorter repairs, more severe faults or no congestion more than double it. The naive planner, which acts on every alarm, loses against doing nothing at the central point, and on intermittent faults every rule does. We repeated every condition with the LLM hybrids and the veto controls (Section 7.6).

7.3. Held-out comparison

Table 3 and Figure 2 report all planners on the three held-out traces. With the per-station detector, the rules recover 13–20% with their default settings; re-tuning on the development trace helps the calibrated rule (+0.27) but not the adaptive ones (+0.14 and +0.18). The naive planner loses against doing nothing under both detectors (−0.25 and −0.28). The

Table 2: Rule-based planners with the per-station detector, held-out traces (216 fault scenarios): 24-h recovery. Central point: $k = 3$, 60 min repairs, $\alpha = 0.5$, contingency mode. Sensitivity rows: all fault shapes, one parameter changed. Default hyperparameters.

	Naive	Calibrated	Adaptive	Twin-only
All fault shapes	−0.28	+0.13	+0.17	−0.20
step	−0.07	+0.23	+0.23	−0.07
intermittent	−1.12	−0.29	−0.27	−0.64
drift	−0.10	+0.22	+0.30	−0.13
$\alpha = 0$	–	+0.37	+0.41	+0.22
$\alpha = 1$	–	−0.07	−0.03	−0.61
$k = 1.5$	–	−1.51	−1.47	−1.75
$k = 5$	–	+0.37	+0.41	+0.03
Repair 15 min	–	+0.54	+0.58	+0.20
Repair 240 min	–	−1.22	−1.22	−1.73
Stop mode	–	−2.48	−2.55	−0.81

burst-veto controls do better than any other rule: +0.27, and +0.34 with the load veto, with half the false repairs of the adaptive rule. Vetoing at random does not help: at the hybrid's veto rate, the adaptive rule recovers +0.15 on average over five seeds (+0.09 to +0.19).

With the per-station detector, the planners separate. The hybrid on Fable 5.1 reaches +0.41, the highest net recovery, while repairing 1.8 healthy stations per scenario and costing 25 piece-min/h without faults, against 5.5 and 89 for the adaptive rule with load veto and 2.8 and 41 for the burst veto with load veto. It acts on the faulty station in fewer scenarios than the rules (87 against 108 of 216). The same hybrid on the 7B model reaches +0.29. Used without the test, the LLM planners do worse: C2 on Fable 5.1 reaches +0.14, below the rule, and C2 on the 7B model −0.25, with 14 false repairs per scenario. Without retrieved knowledge, Fable 5.1 (C1) acts on the faulty station in only 19 scenarios and almost never on a healthy one; the 7B model acts anyway. On intermittent faults the hybrid on Fable 5.1 recovers +0.29, but C1 on Fable 5.1, which almost never acts, also beats every rule without a burst veto there (+0.10); this breakdown is exploratory. Figure 3 places every planner on the trade-off between recovery under faults and the cost of acting on noise.

Scenarios without cost and where the recovery comes from. In 78 of the 216 held-out scenarios the fault costs nothing within 24 h: in 73 the faulty station has no transition in the window, because production is recorded in batches, and in 5 every transition of an intermittent fault falls in its off

Table 3: Held-out traces (216 fault scenarios, 3 fault-free runs), 24-h horizon. Recovery: net, with 95% bootstrap interval stratified by trace. Acts: scenarios in which the faulty station is repaired within 24 h. False repairs: repairs of other stations within 24 h, per scenario. False-alarm cost: piece-min/h in fault-free runs, mean of the three traces.

Planner	Model	Recovery	Step	Interm.	Drift	Acts	False repairs	False-alarm cost
Legacy detector								
Calibrated rule	–	−0.03 [−0.12, +0.04]	+0.04	−0.38	+0.04	62	4.4	108
Adaptive rule	–	−0.03 [−0.14, +0.06]	+0.06	−0.44	+0.06	77	6.2	67
Adaptive + load veto	–	−0.02 [−0.12, +0.07]	+0.08	−0.40	+0.06	75	5.9	64
Twin-only (control)	–	−0.35 [−0.45, −0.27]	−0.26	−0.76	−0.27	47	7.1	142
C1	Qwen 7B	−0.21 [−0.30, −0.12]	−0.13	−0.63	−0.09	49	6.4	136
C2 (RAG)	Qwen 7B	−0.15 [−0.27, −0.05]	−0.06	−0.73	+0.02	104	9.5	159
Hybrid	Qwen 7B	−0.04 [−0.10, +0.03]	−0.02	−0.25	+0.05	51	4.0	38
C2 (RAG)	Fable 5.1	+0.07 [−0.00, +0.14]	+0.13	−0.17	+0.12	54	3.9	58
Hybrid	Fable 5.1	+0.03 [−0.01, +0.08]	+0.09	−0.03	+0.00	24	0.9	7
Per-station detector								
Calibrated rule	–	+0.13 [+0.01, +0.24]	+0.23	−0.29	+0.22	101	5.5	107
Adaptive rule	–	+0.17 [+0.05, +0.27]	+0.23	−0.27	+0.30	108	5.9	98
Adaptive + load veto	–	+0.20 [+0.09, +0.30]	+0.26	−0.20	+0.33	108	5.5	89
Twin-only (control)	–	−0.20 [−0.30, −0.11]	−0.07	−0.64	−0.13	60	6.2	108
Burst veto (control)	–	+0.27 [+0.16, +0.36]	+0.29	−0.07	+0.40	94	4.2	66
Burst + load veto (control)	–	+0.34 [+0.24, +0.43]	+0.32	+0.21	+0.41	91	2.8	41
C1	Qwen 7B	−0.05 [−0.17, +0.06]	+0.06	−0.55	+0.06	90	7.6	121
C2 (RAG)	Qwen 7B	−0.25 [−0.42, −0.10]	−0.05	−1.06	−0.08	117	14.0	203
Hybrid	Qwen 7B	+0.29 [+0.18, +0.38]	+0.32	−0.03	+0.41	106	5.2	74
C1	Fable 5.1	+0.08 [+0.04, +0.14]	+0.02	+0.10	+0.14	19	0.1	1
C2 (RAG)	Fable 5.1	+0.14 [+0.02, +0.24]	+0.23	−0.25	+0.22	105	7.1	100
Hybrid	Fable 5.1	**+0.41** [+0.32, +0.50]	+0.35	+0.29	+0.53	87	**1.8**	**25**

phase. There, any action is pure cost. Table 4 restricts the comparison to the 138 scenarios with cost and splits each planner's delay, as a fraction of the cost of doing nothing, into the fault's own delay left in the window and the delay caused by the planner's actions (contingency processing during repairs and congestion on twins). The ranking does not change: the hybrid on Fable 5.1 recovers +0.44, the adaptive rule with load veto +0.27. But the split shows where the difference comes from. The rule removes more of the fault (it leaves 31% of the fault's delay, against 44% for the hybrid); the hybrid wins because its actions cost a third as much (12% against 42%). C1 on Fable 5.1 marks the other extreme: its actions cost almost nothing, but it leaves 91% of the fault.

7.4. Statistical comparison

Table 5 compares the hybrid on Fable 5.1 with the other planners under the per-station detector. Its closest competitor is the burst-veto control with load veto: the hybrid recovers +0.08 more [+0.01, +0.14], loses less production time in 124 scenarios and more in 19, and is ahead on each held-out trace (+0.07, +0.12 and +0.06) and in all 27 ordered draws of three

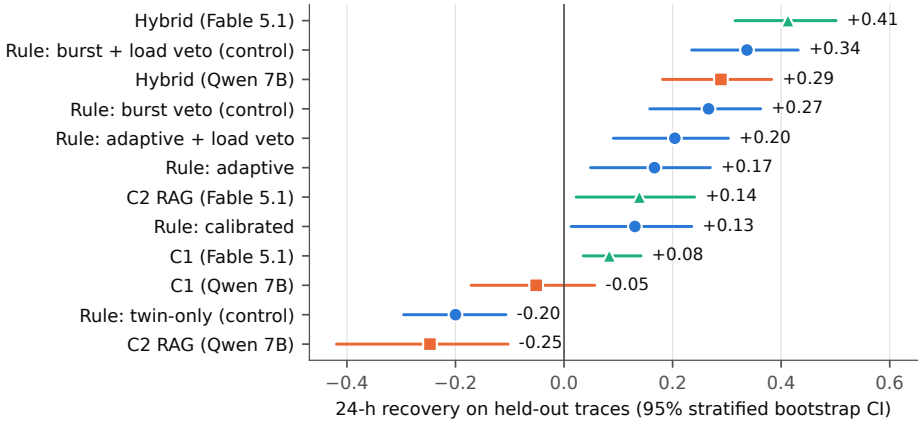


Figure 2: 24-h net recovery on the held-out traces with the per-station detector. Circles: rule-based planners; squares: Qwen2.5 7B; triangles: Claude Fable 5.1.

traces; by shape, the margin is clear only on drift faults. Against the rules without a burst veto the margin is larger: +0.21 [+0.14, +0.28] against the adaptive rule with load veto (139 scenarios better, 18 worse, $r = 0.78$), positive on each trace (+0.23, +0.15 and +0.22), with any trace left out (+0.19 to +0.23), with traces resampled as clusters ([+0.10, +0.28]) and with clusters of trace and station resampled ([+0.13, +0.29]); on the 138 scenarios with cost it is +0.17 [+0.11, +0.24]. It also holds against the rules re-tuned on the development trace (+0.15 [+0.08, +0.22] against the best of them) and against the configuration of the grid that is best on the held-out traces themselves (+0.13 [+0.06, +0.20]), which no honest protocol could select. Against the re-tuned calibrated rule the hybrid gains on intermittent (+0.25 [+0.12, +0.37]) and drift faults (+0.22 [+0.14, +0.32]), and the two are level on step faults (+0.03 [−0.08, +0.15]). The hybrid on the 7B model is ahead of the default rule (+0.08, [+0.05, +0.13] with traces resampled) but level with the best re-tuned rule (+0.02 [−0.02, +0.07]) and behind the burst-veto control with load veto. C2 on Fable 5.1 is behind the rule (−0.07 [−0.17, −0.00] with traces resampled). Wilcoxon tests, which treat scenarios as independent although their windows overlap and they share stored decisions, are in the repository; we rely on the intervals.

Timing, losses and calls. Over a shorter window of 8 h after the onset, the ranking is the same: the hybrid on Fable 5.1 recovers +0.36, the hybrid on the 7B model +0.20, the adaptive rule with load veto +0.11 and C2

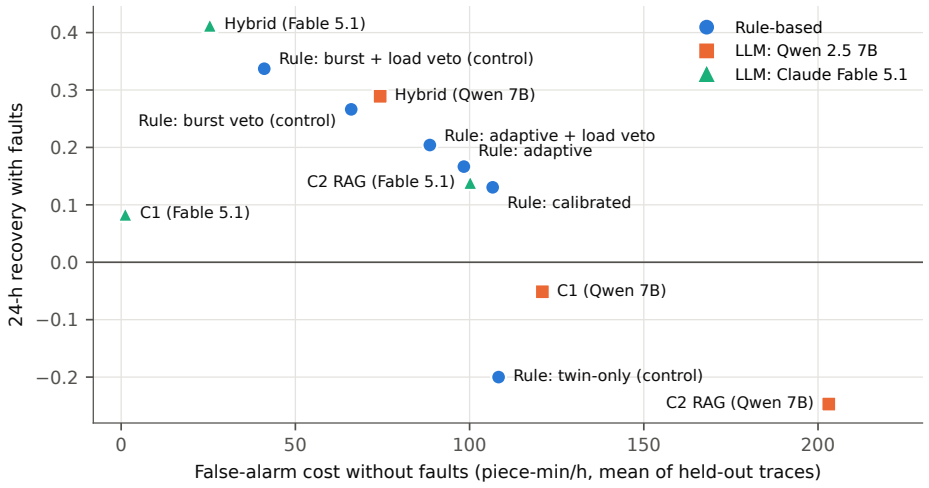


Figure 3: 24-h net recovery with faults against false-alarm cost without faults, held-out traces, per-station detector.

on Fable 5.1 +0.09. The hybrid does not act sooner: when it repairs the faulty station, its median first action comes 345 min after the onset, against 311 min for the rule and 218 min for the calibrated rule. Its advantage is not paid for with large losses against doing nothing: in its worst scenario it adds 2 031 piece-min to the delay of doing nothing (the mean cost of doing nothing is 4 881 piece-min per scenario), against 7 902 for the adaptive rule with load veto. Its largest losses against the rule come from vetoing real faults: in the two worst scenarios, step faults on 200m–300m, the rule removed about 60% of the fault and the hybrid nothing, a difference of 13 941 and 14 508 piece-min. The hybrid calls the model about 20 times per scenario (median 22) and 0.2–0.3 times per hour without faults; C2 calls it 1.0–1.5 times per hour. Charging a fixed cost for every repair, as a maintenance budget would, only widens the gap: at 240 piece-min per repair the hybrid still recovers +0.31 on the held-out traces and +0.35 on the validation traces, while the adaptive rule with load veto falls to −0.09 and −0.19 and the re-tuned calibrated rule recovers +0.17 on the validation traces.

7.5. Validation on new traces

Table 6 reports the five validation traces, generated after every design choice was frozen (Section 6). The hybrid on Fable 5.1 recovers +0.49 [+0.42, +0.55]. The best rule is again the burst-veto control with load veto,

Table 4: The 138 held-out scenarios in which the fault costs something within 24 h, per-station detector. Fault left and action cost are fractions of the cost of doing nothing; recovery = 1− fault left − action cost. Recovery with 95% bootstrap interval stratified by trace.

Planner	Model	Recovery	Fault left	Action cost
Calibrated rule	–	+0.22 [+0.12, +0.32]	0.34	0.43
Adaptive rule	–	+0.23 [+0.13, +0.33]	0.31	0.46
Adaptive + load veto	–	+0.27 [+0.17, +0.36]	0.31	0.42
Twin-only (control)	–	−0.12 [−0.21, −0.03]	0.69	0.43
C1	Qwen 7B	+0.05 [−0.05, +0.14]	0.45	0.50
C2 (RAG)	Qwen 7B	−0.06 [−0.18, +0.05]	0.23	0.83
Hybrid	Qwen 7B	+0.35 [+0.25, +0.44]	0.31	0.34
C1	Fable 5.1	+0.08 [+0.04, +0.14]	0.91	0.01
C2 (RAG)	Fable 5.1	+0.24 [+0.14, +0.33]	0.33	0.44
Hybrid	Fable 5.1	+**0.44** [+0.35, +0.53]	0.44	**0.12**

+0.40 [+0.32, +0.47], and the hybrid is ahead of it by +0.09 [+0.04, +0.15] ([+0.03, +0.20] with traces resampled as clusters, [+0.03, +0.16] with clusters of trace and station), on each of the five traces (+0.05 to +0.29) and in all 3 125 ordered draws of five traces; it loses less time in 212 scenarios and more in 75. Among the rules without a burst veto, the best is the calibrated rule re-tuned on the development trace, +0.37, and the hybrid's margin over it is +0.12 [+0.08, +0.16] (positive on each trace; [+0.07, +0.18] with clusters of trace and station). The hybrid also leads on the 222 scenarios with cost (+0.55 against +0.47 and +0.46). The hybrid on the 7B model does not replicate: +0.23, below both versions of the calibrated rule and both burst controls. [TODO: C2 on Fable 5.1 on the validation traces.]

7.6. When the veto pays off

Table 7 repeats the held-out comparison under every condition of the rules' sensitivity analysis and under common-cause faults, with all planners frozen; the prompt still describes 60-min repairs with backup capacity. The conditions fall into two groups. Where a false repair is expensive (mild faults, 240-min repairs, a stopped line during repairs, full congestion), every rule without a burst veto does much worse than doing nothing or gains little, the burst veto limits the damage, and the hybrid on Fable 5.1 limits it most: it recovers −0.07, −0.09, −0.56 and +0.36, ahead of the burst control with load veto by +0.11 to +0.37 and of the best rule without a burst veto by +0.23 to +0.81. Where acting is cheap (severe faults, 15-min repairs, no congestion), all planners recover 0.4–0.6 and the hybrid is level with the

Table 5: The hybrid planner on Claude Fable 5.1 against other planners, per-station detector, paired over the 216 held-out fault scenarios. Δ: difference in 24-h net recovery (positive favours the hybrid on Fable 5.1), with 95% bootstrap intervals that resample scenarios within their trace and, more conservatively, the 24 clusters of trace and faulty station. r: matched-pairs rank-biserial correlation. Re-tuned rules: selected on the development trace, except the row marked (best on the held-out traces). The breakdown by fault shape is exploratory.

Against	Better / worse	r	Δ [95% CI, by trace]	[95% CI, by cluster]
Burst + load veto (control)	124 / 19	0.64	+0.08 [+0.01, +0.14]	[+0.01, +0.14]
Burst veto (control)	132 / 16	0.74	+0.15 [+0.08, +0.21]	[+0.08, +0.23]
Adaptive + load veto	139 / 18	0.78	+0.21 [+0.14, +0.28]	[+0.13, +0.29]
Adaptive rule	144 / 14	0.81	+0.25 [+0.17, +0.32]	[+0.16, +0.34]
Calibrated rule	138 / 22	0.77	+0.28 [+0.20, +0.37]	[+0.19, +0.40]
Hybrid, Qwen 7B	133 / 25	0.68	+0.12 [+0.06, +0.19]	[+0.05, +0.20]
C2 (RAG), Fable 5.1	141 / 20	0.82	+0.27 [+0.20, +0.35]	[+0.19, +0.37]
Against re-tuned rules				
Calibrated, re-tuned	116 / 37	0.62	+0.15 [+0.08, +0.22]	[+0.07, +0.24]
Adaptive + load veto, re-tuned	143 / 18	0.78	+0.23 [+0.16, +0.30]	[+0.15, +0.32]
Adaptive + load veto, best on held-out	128 / 31	0.66	+0.13 [+0.06, +0.20]	[+0.06, +0.21]
By fault shape, against adaptive + load veto				
step	42 / 7	0.64	+0.09 [−0.02, +0.22]	[−0.03, +0.23]
intermittent	49 / 5	0.85	+0.49 [+0.35, +0.64]	[+0.34, +0.70]
drift	48 / 6	0.83	+0.20 [+0.13, +0.27]	[+0.12, +0.30]
By fault shape, against burst + load veto				
step	38 / 8	0.57	+0.03 [−0.09, +0.15]	[−0.09, +0.16]
intermittent	40 / 9	0.47	+0.08 [−0.03, +0.19]	[−0.02, +0.21]
drift	46 / 2	0.91	+0.12 [+0.06, +0.22]	[+0.07, +0.19]

best rule (−0.03 to +0.02, intervals including zero); with 15-min repairs the hybrid on the 7B model and the plain adaptive rule are slightly ahead of it.

Common-cause faults turn the veto against itself. When one fault degrades four stations at once, their alarms are, by construction, shared bursts; the burst control vetoes them and falls to +0.36, and the hybrid on Fable 5.1 to +0.43, 0.10 below the adaptive rule with load veto ($[-0.17, -0.05]$). The hybrid on the 7B model, which rarely uses the burst argument (Section 7.7), is the best planner there (+0.55). These are 54 scenarios on three traces, and the fault is our construction; but they show that the argument that makes the veto valuable under single-station faults is wrong when the cause really is common.

Three replications of the main hybrid run, each with its own key in the response store so that no response of the original run is reused, recover +0.43, +0.42 and +0.42 (original +0.41), and stay +0.21 to +0.22 ahead of the adaptive rule with load veto; each needed about 330 new calls. Run-to-run variation of the frontier model is small against the differences we report.

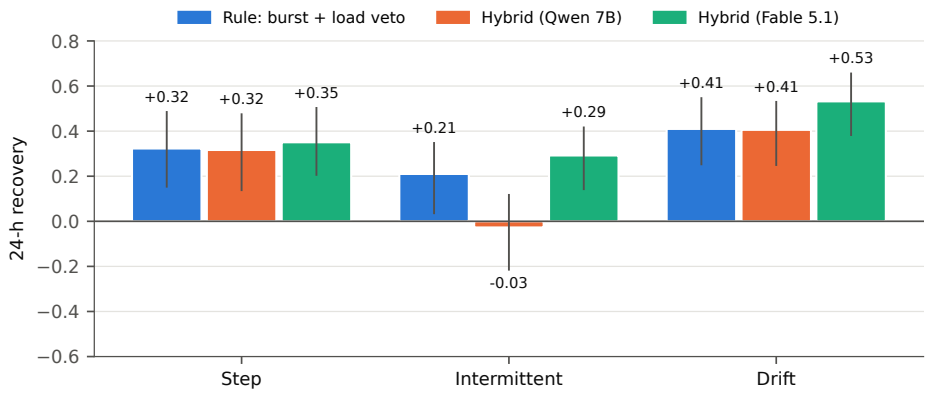


Figure 4: 24-h net recovery by fault shape on the held-out traces, per-station detector (72 scenarios per shape). Whiskers: 95% bootstrap interval stratified by trace.

Table 6: Validation on five new traces (360 fault scenarios), everything frozen: 24-h net recovery with 95% bootstrap interval stratified by trace, and on the 222 scenarios with cost. Base rates from the development trace.

Planner	Model	Recovery	With cost	Intermittent	Δ vs hybrid (Fable 5.1)
Calibrated rule	–	+0.27 [+0.19, +0.35]	+0.40	−0.31	−0.21 [−0.26, −0.17]
re-tuned	–	+0.37 [+0.30, +0.44]	+0.46	−0.05	−0.12 [−0.16, −0.08]
Adaptive rule	–	+0.10 [−0.00, +0.19]	+0.29	−0.64	–
Adaptive + load veto	–	+0.17 [+0.07, +0.25]	+0.34	−0.51	−0.32 [−0.40, −0.26]
re-tuned	–	+0.19 [+0.09, +0.27]	+0.35	−0.49	−0.30 [−0.37, −0.24]
Twin-only (control)	–	−0.31 [−0.41, −0.23]	−0.15	−1.03	–
Burst veto (control)	–	+0.33 [+0.25, +0.40]	+0.42	−0.20	−0.16 [−0.22, −0.10]
Burst + load veto (control)	–	+0.40 [+0.32, +0.47]	+0.47	+0.05	−0.09 [−0.15, −0.04]
Hybrid	Qwen 7B	+0.23 [+0.14, +0.31]	+0.38	−0.35	−0.26 [−0.32, −0.19]
Hybrid	Fable 5.1	**+0.49** [+0.42, +0.55]	**+0.55**	+0.15	–

7.7. What the vetoes rest on

The hybrid on Fable 5.1 is the most conservative planner under the legacy detector and the best one under the per-station detector. Its stated reasons barely change: coded with an explicit keyword rule (*burst*, *line-wide*, *common cause*, *systemic*; rule and counts in the repository; a broader rule that adds words such as *shared* and *upstream* gives 99% and 94%), 99% of its vetoes argue that the alarms come in bursts shared by many stations under the legacy detector (1 596 of the 1 607 calls with a veto) and 91% under the per-station detector (2 587 of 2 833); counted over distinct responses rather than calls, 97% and 88%. Besides the alarms themselves, two things could explain this: the paragraph of the prompt that describes the detector changes between the two conditions, and the hybrid's prompt names such bursts among the reasons to doubt the test. Table 8 separates them.

Table 7: Sensitivity of the planners, held-out traces, per-station detector: 24-h net recovery. One parameter changed at a time from the central point ($k = 3$, 60-min repairs, $\alpha = 0.5$, contingency mode); last row: the same fault on the four stations with a twin, or on the four without, at once (54 scenarios). Rules with default settings except the re-tuned calibrated rule. Δ: hybrid on Fable 5.1 minus the burst control with load veto, 95% bootstrap interval stratified by trace.

Condition	Adaptive + load veto	Calibrated, re-tuned	Burst + load veto	Hybrid, Qwen 7B	Hybrid, Fable 5.1	Δ [95% CI]
Central	+0.20	+0.27	+0.34	+0.29	**+0.41**	+0.08 [+0.01, +0.14]
False repairs expensive						
$k = 1.5$	−1.30	−0.79	−0.44	−0.95	**−0.07**	+0.37 [+0.29, +0.46]
Repair 240 min	−0.86	−0.81	−0.20	−0.90	**−0.09**	+0.11 [+0.02, +0.20]
Stop mode	−1.75	−1.40	−0.76	−1.86	**−0.56**	+0.20 [+0.11, +0.30]
$\alpha = 1$	−0.01	+0.13	+0.23	+0.18	**+0.36**	+0.13 [+0.06, +0.20]
Acting cheap						
$k = 5$	+0.43	+0.43	+0.47	+0.47	**+0.50**	+0.02 [−0.04, +0.08]
Repair 15 min	+0.58	+0.53	+0.53	**+0.59**	+0.56	+0.03 [−0.01, +0.08]
$\alpha = 0$	**+0.46**	+0.43	+0.45	+0.40	**+0.46**	+0.01 [−0.05, +0.07]
Common cause	+0.53	+0.51	+0.36	**+0.55**	+0.43	+0.07 [−0.02, +0.15]

Table 8: Ablations of the hybrid planner on Claude Fable 5.1, held-out traces, 24-h net recovery with 95% bootstrap interval stratified by trace. Alarm text: the paragraph of the prompt that describes the detector (v2: legacy; v3: per-station). Burst cue: the hybrid prompt's mention of "a burst of alarms shared by many stations at once" among the reasons to doubt the test. First row: the configuration of Tables 3–5.

Detector	Alarm text	Burst cue	Recovery	Calls with a veto	Vetoes citing bursts
Per-station	v3	yes	+0.41 [+0.32, +0.50]	2 833	91%
Per-station	v2	yes	+0.41 [+0.32, +0.51]	2 601	88%
Per-station	v3	no	+0.31 [+0.21, +0.41]	1 409	44%
Legacy	v2	yes	+0.03 [−0.01, +0.08]	1 607	99%
Legacy	v3	yes	+0.03 [−0.00, +0.07]	1 784	97%

The description of the detector does not change the aggregate recovery: with either detector, swapping the alarm paragraph leaves it unchanged (-0.00 $[-0.05, +0.03]$ and $+0.00$ $[-0.03, +0.03]$). It does change decisions: under the per-station detector, the v3 text loses less time in 83 scenarios and more in 8, and the per-trace differences (+0.03, −0.06, −0.00) cancel; the intervals do not show that the two texts are equivalent, only that neither is better overall. The structure of the alarms matters more: with the same prompt, the hybrid recovers +0.03 on the legacy detector's bursts and +0.41 on per-station alarms. The burst cue helps: without it, the hybrid vetoes in half as many calls, cites bursts in 44% of its vetoes instead of 91%, and recovers 0.10 less ($[+0.07, +0.13]$, positive on each trace).

Table 9: Share of the change test's candidates that each hybrid vetoes, by the number of stations named in the alarm behind the call, from the fault onset on; held-out traces, per-station detector. In parentheses: number of candidates. Recomputed by replaying the stored decisions.

	Fable 5.1		Qwen 7B	
Stations in the alarm	faulty	healthy	faulty	healthy
1	17% (66)	40% (329)	2% (44)	17% (298)
2	32% (25)	64% (319)	3% (29)	11% (199)
3	57% (23)	93% (314)	6% (18)	17% (266)
4	38% (8)	91% (172)	0% (12)	8% (165)
5 or more	88% (17)	91% (363)	26% (19)	16% (385)

Table 9 shows what the veto responds to. The hybrid on Fable 5.1 vetoes a candidate more often the more stations the alarm names, as a burst rule would; but at every alarm size it vetoes the faulty station less often than a healthy one (17% against 40% in single-station alarms, 57% against 93% in alarms of three). That difference is what the burst control cannot reproduce. The hybrid on the 7B model vetoes little at any size and barely separates faulty from healthy stations; it uses the burst argument in 0.4% of its vetoes under the legacy detector and in none under the per-station detector by the strict coding rule, but in 43% under the legacy detector by the broader one, so that statement depends on the coding.

7.8. Why the evaluation design matters

In a first phase of this research, every fault was injected 600 min after the first event of a trace, a repair cleared any fault that began during it, and recovery was measured over the whole trace. Under that design the legacy detector appeared to work: the adaptive rules recovered 52–54%, C2 on Fable 5.1 58% and the hybrid on Fable 5.1 24% (Table 10). Those gains were an artefact of the design. The fixed onset fell inside the detector's start-up burst of alarms and right after the 8-h warm-up of the planners' memory, when C2 repairs many stations at once; routine repairs then coincided with the fault or preceded it by minutes and, under the old repair rule, cleared it at no cost. Moving onsets out of the start-up period, letting repairs clear only active faults and bounding the metric to 24 h removed those gains; the ranking of the two LLM designs reversed only with the per-station detector (under the legacy detector, C2 on Fable 5.1 stays ahead of the hybrid, +0.07 against +0.03). We report the first design because the error is easy to make: a closed-loop controller evaluated at a single, fixed fault time inherits whatever happens to the line at that time.

Table 10: The same planners under the first evaluation design (v1: fault at 600 min, a repair clears faults that begin during it, recovery over the whole trace, 72 scenarios) and the present one (v2: three random onsets after the start-up period, a repair clears only active faults, 24-h net recovery, 216 scenarios). Held-out traces.

Planner	v1, legacy	v2, legacy	v2, per-station
Adaptive + load veto	+0.54	−0.02	+0.20
C2 (RAG), Qwen 7B	+0.20	−0.15	−0.25
Hybrid, Qwen 7B	+0.41	−0.04	+0.29
C2 (RAG), Fable 5.1	+0.58	+0.07	+0.14
Hybrid, Fable 5.1	+0.24	+0.03	+0.41

8. Discussion

The Analyze layer and the planner interact. Closing the original loop [3] on its own detector recovers almost nothing within a day: no planner recovers more than 0.07, because the detector reports a fault late, rarely and inside line-wide bursts. Replacing it with a per-station test on processing times moves the best rule from −0.02 to +0.20 and the best planner from +0.07 to +0.41. But the planner matters as much once the alarms are useful: under the per-station detector the planners span 0.66, from −0.25 to +0.41. The Analyze layer sets what a planner can achieve; the planner decides how much of it is achieved.

Propose and veto: the value is in not acting. Given useful alarms, the LLM adds most as a veto over a statistical test, and it does so by declining to act, not by detecting more. The split of the delay shows the effect: the rule removes more of the fault, but its false repairs cost more than the extra fault it removes; the hybrid leaves more of the fault and pays a third of the cost. Most of this is a simple argument that a rule can implement: do not repair a station whose alarm is shared by many others. A rule that does only that, with the load veto, reproduces about two thirds of the hybrid's margin over the adaptive rule (from +0.20 to +0.34 on the held-out traces, from +0.17 to +0.40 on the validation traces). What the LLM adds beyond it, +0.08 and +0.09, comes from applying the argument selectively: at every alarm size it vetoes the faulty station less often than a healthy one (Table 9). Declining alone is not enough: vetoing at random does not help, and C1 on the frontier model, without the test or retrieved knowledge, barely acts and recovers little. On their own, both LLM planners do worse than the rule.

The veto is worth what a false repair costs. The sensitivity analysis gives the result its scope (Table 7). Where false repairs are expensive (mild faults, long repairs, a stopped line, full congestion), the hybrid on Fable 5.1 is by far the best planner, often the only one that does not do much more harm than good. Where acting is cheap (severe faults, short repairs, no congestion), every reasonable planner recovers about half of the delay and the LLM adds nothing. And when faults really do have a common cause, the burst argument discards them: the hybrid falls behind the plain rule, and the 7B hybrid, which does not use the argument, does best. The value of an LLM veto is therefore not a property of the model but of the plant: it depends on the cost of an unnecessary intervention and on whether alarms shared by several stations are, there, usually noise or usually a common fault.

The model matters inside the right architecture. The same hybrid recovers +0.41 on Fable 5.1 and +0.29 on the 7B model on the held-out traces, and +0.49 against +0.23 on the new traces, where the 7B hybrid falls below the best rules. Only the frontier model's advantage replicates. This is a contrast between two systems, which differ in size, quantisation, provider and inference settings, not a measured capability curve.

Implications for practice. Within the limits of a study with synthetic faults, four points transfer to automation architects and maintenance engineers. First, connecting threshold alarms directly to maintenance orders is harmful: the naive planner loses against doing nothing under both detectors, because false repairs accumulate; alarms must be rationalised before they drive actions, by analogy with what alarm-management practice requires for operators [13, 14]. Second, a detector should compare each station with its own recent behaviour on the part of the signal a fault affects, here processing times rather than storage-dominated transitions, and with a test that controls its false alarms. Third, whether acting pays off depends on the severity of faults relative to the cost of a repair and on backup capacity during repairs; before adding an LLM, a plant should try a rule that suppresses actions on alarms shared by many stations, which here recovers most of what the LLM does, and should check how often its shared alarms are in fact common-cause faults. Fourth, an LLM veto is worth its cost where unnecessary interventions are expensive; its non-determinism, its sensitivity to how alarms are structured and its failure under common-cause faults argue for keeping every action behind a symbolic validator, as here, and, in a plant, behind human approval in the maintenance management system that

issues the work orders.

Development sets mislead. The per-station detector and the re-tuned rules were chosen on the development trace, where the detector names the faulty station in all 72 scenarios and the adaptive rule recovers +0.84; on the held-out traces the detector names it in 122 of 216 and the same rule recovers +0.20, and the configurations re-tuned there help one rule and not the others. A single trace is too small to select designs on, and LLM-based controllers should be evaluated on data not used to write their prompts, with fault onsets that do not coincide with the controller's start-up.

Threats to validity. Construct: faults are synthetic, and the effect model is additive and trace-driven, without queueing, so every recovery is a property of the model, not a measured property of the Bosch line; its parameters were varied one at a time for all planners (Table 7), but not jointly, and the frozen prompt describes 60-min repairs with backup capacity in every condition. Common-cause faults were tested in one form only (one fault on four stations at once, 54 scenarios). Recovery is net of the cost of acting, measured in production time within 24 h of the fault; a fixed cost per repair, which we tried as a check, only widens the hybrid's lead, but we use no economic weights. A 24-h window is chronological, not productive time, and in about a third of the scenarios the fault costs nothing in it (Table 4). The per-station detector's reference is the station's previous 24 h, so a persistent fault is absorbed into it after about a day. Twins are structural, not verified equipment equivalents. *Internal*: the held-out traces informed the decision to build a new detector and the evaluation design, though not their parameters (Section 6); the validation traces were generated after everything was frozen. Each frontier-model decision is a stored sample at low effort, and most calls reuse a stored response, which also makes scenarios share decisions; three replications without the stored responses changed the main recovery by at most 0.014. Prompt v3 was tested on the development trace with the 7B model only; the prompts are part of the treatment; the control rules cover the load veto and the burst argument but not every statistic C2 retrieves, such as the outcome of past repairs; the 7B model is 4-bit quantised. The frontier model may have seen our earlier papers [3, 4], which name the Bosch line, but the faults are synthetic and not described anywhere it could have seen. *External*: one production line, one dataset, the test-set pieces only; two language models, with no intermediate sizes. *Statistical*: the 24-h windows of scenarios on the same trace overlap; the held-out set has three traces and the validation set five, and resampling

traces or clusters of trace and station addresses the dependence only partly.

9. Conclusion

We closed the autonomic loop of a self-coordinated manufacturing architecture that, as published, only detected. On its own detector, no planner recovers delay: the detector raises line-wide bursts of alarms and reports a faulty station late and rarely. With a per-station change test on processing times, and within our effect model, a frontier LLM with retrieved operational statistics that vetoes the test's candidates recovers about two fifths of the delay within a day on the held-out traces and half on five traces generated after the design was frozen, ahead of every rule we tried, and three replications reproduce it. But a plain rule that vetoes alarms shared by many stations gets most of the way there, and the LLM's remaining margin, 8–9 points, comes from applying that argument selectively. The veto is worth much where false repairs are expensive, nothing where acting is cheap, and it hurts when faults have a common cause. Used on its own, the same model does not beat the test, and the 7B model's advantage does not replicate. An earlier evaluation, with a fixed fault time, had suggested the opposite conclusions. Future work will validate the effect model against plant data with real capacities, repair times and common-cause faults, add intermediate model sizes, and test the detector on faults that persist beyond a day.

Appendix A. Prompts

System prompt v3 of C1, as sent to both models (exported from the code):

```
You are the Plan phase of an autonomic controller for a manufacturing line
     (the Bosch production line: about 50 stations on 4 sub-lines).

How alarms work. The Analyze phase checks each station separately. It
    raises an alarm listing the stations whose recent processing times (
    transitions of up to 60 minutes leaving the station) exceed the
    station's own recent history more often than chance would explain. In
    normal operation this detector also raises false alarms: a station
    named in an alarm may be perfectly healthy.

What you can do for each flagged station:
- "none": leave it alone;
- "maintain": repair it for 60 minutes. While it is being repaired a
    slower backup handles its work. Repairing a degraded station fixes it.
     Repairing a healthy station fixes nothing and only slows production;
```

```
- "reroute_and_maintain": only for a station with a free twin. The twin
    takes all of the station's load during the repair (the twin slows down
     because of the extra load), and the station is repaired.

Objective: minimise total production delay. Acting on a healthy station
    wastes production. Ignoring a truly degraded station lets its delay
    accumulate for as long as it stays degraded.

For EVERY station listed, first give a verdict, then an action:
- "degraded": the evidence shows the station itself is not working
    properly;
- "normal": the station is probably healthy and the alarm is a false alarm
    ;
- "self_inflicted": the station is alarming because of load you rerouted
    onto it;
- "insufficient_evidence": you cannot tell.
Only a "degraded" station may receive an action other than "none".

Answer only with JSON matching the schema. Keep "reason" under 30 words.
```

C2 appends:

```
You also receive RETRIEVED KNOWLEDGE for each station, computed from your
    operational log:
- how many alarms the station raised recently, and how many its normal (
    fault-free) alarm rate would predict over the same period;
- whether you are currently pushing extra load onto it;
- what happened to its alarms after you repaired it before;
- its recent log records.
How to read it:
- many more alarms than expected, sustained over the last 24 h, points to
    "degraded";
- about as many alarms as expected points to "normal";
- a station receiving load you rerouted onto it is "self_inflicted":
    repairing it does not help;
- if alarms did not drop after a previous repair, the alarms were probably
     not caused by a fault;
- with too little history, prefer "insufficient_evidence".
```

The hybrid appends, after C2's text:

```
In this mode a statistical change detector has already screened the alarms
    . The stations listed were flagged because their recent alarm count is
     improbably high compared with their own recent history. The detector
    is usually right, but it cannot see context: load you rerouted onto a
    station, a previous repair that did not reduce its alarms, a burst of
    alarms shared by many stations at once. Give "degraded" unless the
    retrieved knowledge gives a concrete reason to doubt the detector;
    then give the verdict that reason supports.
```

Prompt v2, used with the legacy detector, replaces the first sentence of the second paragraph of the C1 prompt (from "How alarms work" to "explain.") with:

```
How alarms work. The Analyze phase raises an alarm listing stations whose
    recent transition times exceeded 1.3 times their reference time within
     the same batch of events.
```

The user message lists the time, the stations to judge with the state of their twins, the stations under repair and the active reroutes; for C2 and the hybrid it adds, per station, the retrieved knowledge described in Section 5. The JSON schema of the answer is in the repository.

Data and code availability

The headless simulator, planners, prompts, development log, every model decision with its stated reason, the results and the scripts that produce every table and figure are available at `https://github.com/mbsanchez/ssacsimulator`. The simulator uses the Python standard library only; the statistics and figures additionally use SciPy and matplotlib. The Bosch dataset is publicly available [18]; the files derived from it (process model and event traces) are not redistributed, and the repository gives the scripts that produce them, including the Python 3 port of the original preprocessing used for the five validation traces, and their checksums.

CRediT authorship contribution statement

Manuel Sánchez: Conceptualization, Methodology, Investigation, Visualization, Writing – original draft. **Ernesto Exposito:** Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) to assist in developing the simulation and analysis software, to draft and

revise the text of the manuscript, and as an adversarial agent acting as a peer reviewer to identify weaknesses and suggest improvements. The software was reviewed by the first author. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article. The use of Qwen2.5 and Claude Fable 5.1 as planners is part of the method studied in this paper (Section 5) and is unrelated to the preparation of the manuscript.

References

[1] J. O. Kephart, D. M. Chess, The vision of autonomic computing, Computer 36 (1) (2003) 41–50. doi:10.1109/MC.2003.1160055.

[2] D. Weyns, An Introduction to Self-adaptive Systems: A Contemporary Software Engineering Perspective, Wiley-IEEE Press, 2021. doi:10.1002/9781119574910.

[3] M. Sánchez, E. Exposito, J. Aguilar, Implementing self-* autonomic properties in self-coordinated manufacturing processes for the Industry 4.0 context, Computers in Industry 121 (2020) 103247. doi:10.1016/j.compind.2020.103247.

[4] M. Sánchez, E. Exposito, J. Aguilar, Autonomic computing in manufacturing process coordination in Industry 4.0 context, Journal of Industrial Information Integration 19 (2020) 100159. doi:10.1016/j.jii.2020.100159.

[5] Y. Xia, M. Shenoy, N. Jazdi, M. Weyrich, Towards autonomous system: flexible modular production system enhanced with large language model agents, in: 2023 IEEE 28th International Conference on Emerging Technologies and Factory Automation (ETFA), 2023, pp. 1–8. doi:10.1109/ETFA54631.2023.10275362.

[6] M. Li, Q. Zhou, W. Li, T. Qu, M. Yang, P. Jiang, A4PS: Agentic AI-assisted advanced planning and scheduling with large language models for smart manufacturing, Journal of Manufacturing Systems 85 (2026) 207–226. doi:10.1016/j.jmsy.2026.01.003.

[7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in: Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 9459–9474.

[8] J. Li, M. Zhang, N. Li, D. Weyns, Z. Jin, K. Tei, Generative AI for self-adaptive systems: State of the art and research roadmap, ACM Transactions on Autonomous and Adaptive Systems 19 (3) (2024) 13:1–13:60. doi:10.1145/3686803.

[9] J. Li, M. Zhang, N. Li, D. Weyns, Z. Jin, K. Tei, Exploring the potential of large language models in self-adaptive systems, in: Proceedings of the 19th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS), 2024, pp. 77–83. doi:10.1145/3643915.3644088.

[10] N. Nascimento, P. Alencar, D. Cowan, Self-adaptive large language model (LLM)-based multiagent systems, in: 2023 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C), 2023, pp. 104–109. doi:10.1109/ACSOS-C58168.2023.00048.

[11] L. Alves, G. L. B. De Oliveira, M. Chagas, A. Bandeira, P. H. Maia, C. Rocha, MAPER: Extending MAPE-K with LLM-based reasoning to manage unanticipated situations in self-adaptive systems, in: Proceedings of the 21st International Conference on Software Engineering for Adaptive and Self-Managing Systems (SEAMS), 2026, pp. 239–250. doi:10.1145/3788550.3794886.

[12] M. Sánchez, E. Exposito, J. Aguilar, Industry 4.0: survey from a system integration perspective, International Journal of Computer Integrated Manufacturing 33 (10–11) (2020) 1017–1041. doi:10.1080/0951192X.2020.1775295.

[13] International Society of Automation, ANSI/ISA-18.2-2016: Management of Alarm Systems for the Process Industries, International Society of Automation, Research Triangle Park, NC, american National Standard (2016).

[14] International Society of Automation, ISA-TR18.2.6-2012: Alarm systems for batch and discrete processes, Tech. rep., International Society of Automation, Research Triangle Park, NC (2012).

[15] M. Basseville, I. V. Nikiforov, Detection of Abrupt Changes: Theory and Application, Prentice Hall, 1993.

[16] N. D. Thuan, L. H. Anh, Large language models for fault diagnosis in smart manufacturing: A comprehensive review, Intelligent Systems with Applications 31 (2026) 200714. doi:10.1016/j.iswa.2026.200714.

[17] M. Sadeqi Bajestani, D. Mun, D. B. Kim, Human-in-the-loop and large language models in smart manufacturing: Current applications, challenges, and perspectives, Journal of Manufacturing Systems 86 (2026) 913–941. doi:10.1016/j.jmsy.2026.04.027.

[18] Bosch, Bosch production line performance, Kaggle competition dataset, https://www.kaggle.com/c/bosch-production-line-performance (2016).

[19] Qwen Team, Qwen2.5 technical report, arXiv:2412.15115 (2024).

[20] Ollama, Ollama: run large language models locally, https://ollama.com, version 0.35.1 (2026).

[21] Anthropic, Claude agent sdk, https://code.claude.com/docs/en/agent-sdk, python package claude-agent-sdk 0.2.163 (2026).

[22] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, 1993.

[23] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.

[24] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.